

would be at a single place.

Disadvantages

1. We would have to write separate logic for handling streaming only audit events as these audit events are not stored in the database and therefore the hooks won't get triggered for such audit events.
1. In case of any changes in the existing business logic to audit events will also require changes to the checks.
1. We also plan to move audit events to ClickHouse which might require re-work on this feature as we don't know how callbacks or database triggers would work after the migration to ClickHouse.

Use Sidekiq workers for creating and updating Checks

In this approach we completely decoupled audit events with the checks. The checks would get created via Sidekiq workers in the background.

These workers would get enqueued whenever a related setting is toggled.

Consider the above example, we would enqueue the Sidekiq worker responsible for creating checks whenever the setting related to merge request approvals is updated. The sidekiq worker would fetch the updated value of merge request approval setting in the background and would create or update the result of this check for the given project in the database.

Advantages

1. This is a modular approach thus allowing us to modify the business logic independently.
1. Changes to the implementation of audit events will not impact these checks.

Disadvantages

1. Maintenance of additional background worker. In case of Redis failures we could miss events, however, we do have the option of refreshing check status for all the projects in a group.
1. Audit events cannot be used as the single source of truth for these checks, however, a user can correlate the status of the check with the audit events.

Decision

The decision was made to use sidekiq workers for creating results of checks for a project.

The database table for storing the results is projectcompliancestandardsadherence and has the

following
schema:

```
mermaid
classDiagram
    class projectcompliancestandardsadherence {
        id bigint
        createdat timestamp
        updatedat timestamp
        projectid bigint
        namespaceid bigint
        checkname smallint
        status smallint
    }
```

checkname is Enum and contains values like "preventapprovalbymergerequestauthor", "preventapprovalbymergerequestcommitters", "atleasttwoapprovals", etc.

status can have values "success" or "fail" denoting the status of that compliance adherence check for a project.

This table is used to display the status of checks for projects on the compliance adherence dashboard.

SECTION: engineering/architecture/design-documents/compliance-
frameworks/decisions/002_custom_adherence_report.md

Context

We delivered compliance standards adherence report MVC. This was well received by our customers, however, we received feedbacks around the possibility of customising the compliance adherence dashboard. The existing compliance standards adherence report displays all the checks that are currently supported and displays the compliance status of all the projects within that group. Currently, users cannot configure this behaviour.

Approach

To allow customising the compliance standards adherence report we plan to introduce the concept of compliance requirements and compliance checks.

1. Compliance Requirement: A compliance framework can have multiple compliance requirements. These requirements can have multiple checks associated to it. Requirement allows the users to club multiple checks together as per their need.

For example: A requirement could be "Code safeguards" and it can contain the "At least two approvers" and

"Prevent authors as approvers" checks.

1. Compliance Check: They belong to a compliance requirement and denotes one of the checks supported by GitLab product.

We thought of the following approaches to implement custom compliance adherence report

1. Using YAML for storing compliance adherence configuration
2. Storing the compliance adherence configuration in database as JSON
3. Storing the compliance adherence configuration in database as relational data (selected approach)

Using YAML for storing compliance adherence configuration

1. Similar to security policies, we planned on using YAML files for storing the compliance adherence report configuration.

1. Users would create a file for each framework named `.compliance-standard@frameworkname.yml`. These YAML files

would be stored in special projects within that group similar to security policies project

which is used for storing security policies.

1. While creating compliance frameworks users would be able to set the reference of this standards YAML file to associate the YAML with that compliance framework.

1. We would create a Standards Editor similar to Policy Editor easily allowing users to modify the configuration from the UI.

1. The YAML would be used for saving the requirements and checks configuration. An example YAML:

```
yaml
requirements:
- name: Code safeguards enabled
  description: Ensure that code is guarded correctly.
  checks:
    - atleasttwoapprovals
    - preventapprovalbymergerequestauthor
- name: Authentication Control
  description: Ensure all users have multi-factor authentication and use organisations
email.
  checks:
    - multifactorauthenabled
    - organizationsemaildomain
```

1. The checks would be a list of Enums that and denotes one of the checks supported by GitLab.

1. Once the configuration is saved and applied to a compliance framework, we would only display the results of the

selected checks for the projects with those compliance frameworks.

1. We would not display the compliance adherence status for projects that don't have a compliance framework applied.
1. This approach had certain advantages:
 1. Compliance configurations stored as code, easier to share and reuse it.
 1. This would also allow in the future for some kind of centralized registry in GitLab for these configurations.
 1. Standards YAML configured through merge requests helps in auditing and viewing historical changes easily.
1. This approach also had several disadvantages:
 1. It seems to be a bit complex to adopt for compliance users.
 1. Accessing the configuration via the repository might be a bit slower as compared to storing it in database.
 1. YAML structure needs to be designed properly to allow any future changes to the YAML schema easily.

Storing the compliance adherence configuration in database as JSON

1. In this approach we planned to store the compliance adherence report configuration as a JSON column in the compliance frameworks table.
1. We would create a UI for creating requirements within compliance frameworks and also adding checks to this requirement.
1. We would translate this configuration into a JSON and store it in the jsonb column named requirements within the compliance frameworks table.
1. An example JSON:

```

json
[
  {
    "name": "Code safeguards enabled",
    "description": "Ensure that code is guarded correctly",
    "checks": [
      "atleasttwoapprovals",
      "preventapprovalbymergerequestauthor"
    ]
  },
  {
    "name": "Authentication Control",
    "description": "Ensure all users have multi-factor authentication and use organisations email",
    "checks": [
      "multifactorauthenabled",
      "organizationsemaildomain"
    ]
  }
]

```

1. Similar to the previous approach, we would only display the results of checks for the projects that have compliance

frameworks applied with valid requirements configuration.

1. This approach had certain advantages:

1. This gives a UI friendly approach.

1. No need for setting up special projects and understand complex YAML configurations.

1. No major database changes required.

1. This approach also had disadvantages:

1. Fetching the requirements JSON from the database will not be an easy operation.

1. Designing a scalable JSON schema validator that will also work easily for future iterations might be complex.

1. Viewing historical changes to the configuration will not be easy. Users will have to track changes through

historical audit event logs.

1. Updating the existing JSON configurations will be difficult when compared to storing it as a non JSON column.

Storing the compliance adherence configuration in database as relational data

1. In this approach we planned to store the configuration in new database tables to prevent issues with JSON columns.

1. The compliance requirements and checks would be stored in individual tables with the following schema:

```
mermaid
classDiagram
class namespaces {
    id: bigint
    name: text
    path: text
    ...(more columns)
}
class projects {
    id: bigint,
    name: text
    path: text
    description: text
    ...(more columns)
}
class compliancemanagementframeworks {
    id: bigint,
    name: text,
    description: text,
    ...(more columns)
}

class compliancerequirements {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    namespaceid: bigint
    frameworkid: bigint
}
```

```

    name: text
    description: text
  }

class compliancechecks {
  id: bigint
  createdat: timestamp
  updatedat: timestamp
  requirementid: bigint
  namespaceid: bigint
  checkname: smallint
}

class projectcompliancestandardsadherence {
  id: bigint
  createdat: timestamp
  updatedat: timestamp
  projectid: bigint
  namespaceid: bigint
  checkname: smallint
  status: smallint
}

compliancerequirements --> compliancechecks : hasmany
compliancerequirements <-- compliancechecks : belongsto
compliancemanagementframeworks --> compliancerequirements : hasmany
compliancemanagementframeworks <-- compliancerequirements : belongsto
compliancemanagementframeworks <--> projects : manytomany
projects <-- namespaces : hasmany
projects --> namespaces : belongsto
namespaces --> compliancemanagementframeworks : hasmany
namespaces <-- compliancemanagementframeworks : belongsto
projects --> projectcompliancestandardsadherence : hasmany
projects <-- projectcompliancestandardsadherence : belongsto

```

1. We have dropped the standard column from the projectcompliancestandardsadherence since we don't want to associate checks with a standard anymore, therefore, allowing the users to customise and group checks as per their requirements.

1. Similar to the previous approaches, we would still be storing results for all the compliance adherence checks for ultimate projects, irrespective of the frameworks applied to that project, however, we would only display the results of checks for the projects that have compliance frameworks applied with requirements configured at the compliance standards adherence dashboard. This means there is no need for a relationship between compliancechecks and projectcompliancestandardsadherence database tables.

1. In the next iteration we would also allow importing and exporting the compliance adherence report configurations.

Decision

We went ahead with storing the compliance adherence configuration in database as relational data.

SECTION: engineering/architecture/design-documents/compliance-frameworks/decisions/003_custom_controls.md

Context

We want the ability to create custom requirements so that users don't need to rely only on the exhaustive list of controls that GitLab supports or would support in the future. Users could also have external services that might impact whether their projects are compliant to a standard or not.

Approach

To allow users to create controls on their own as per their requirements we need to have the following types of requirements:

1. Internal requirements: Enable users to create logical expressions with all available project settings.
1. External requirements: Enable users to create requirements that rely on their external services like HTTP servers.

Internal requirements

We would allow users to create logical expressions with all the available project settings. These expressions would be the controls against which the projects would be evaluated. We would store these as a structured JSON in the `compliance_requirements` table with 'internal' as the requirement type.

As an example: Consider expression 'mergemethod' = 'merge commit' AND ('projectname' LIKE "%-team" OR 'complianceframework' != 'SOC2').

This expression will be saved in the database as JSON with the following structure:

```
json
{
  "operator": "AND",
  "conditions": [
    {
      "field": "mergemethod",
```

```

    "operator": "=",
    "value": "merge commit"
  },
  {
    "operator": "OR",
    "conditions": [
      {
        "field": "projectname",
        "operator": "LIKE",
        "value": "%-team"
      },
      {
        "field": "complianceframework",
        "operator": "!=",
        "value": "SOC2"
      }
    ]
  }
]
}

```

We can also store simple expressions like `mergemethod = 'merge commit'`, this would be stored in the database as the following JSON:

```

json
{
  "operator": "=",
  "field": "mergemethod",
  "value": "merge commit"
}

```

We would create schema validators for validating the input and store these in the `expression jsonb` column of the `compliancerequirements` database table.

We would have a UI on the frontend with dropdowns to choose the field, operator and values. This is created so that the users don't have to write complex JSON expressions on their own.

The above JSON would be parsed using an evaluator and the expression would be evaluated to true or false. A basic evaluator class would look like:

```

ruby
module ComplianceManagement
  module ComplianceRequirement
    class QueryEvaluator

```



```

def initialize(query, project)
  @query = query
  @project = project
end

def evaluate
  evaluatenode(@query)
end

private

def evaluatenode(node)
  if node['operator']
    evaluateoperatornode(node)
  else
    evaluatecondition(node)
  end
end

def evaluateoperatornode(node)
  operator = node['operator'].upcase
  conditions = node['conditions']

  Handle the case of a single condition
  if conditions.nil? || conditions.empty?
    return evaluatecondition(node)
  end

  results = conditions.map { |condition| evaluatenode(condition) }

  case operator
  when 'AND'
    results.all?
  when 'OR'
    results.any?
  else
    raise "Unknown operator: {operator}"
  end
end

def evaluatecondition(condition)
  fieldvalue = getfieldvalue(condition['field'])

  case condition['operator']
  when '='
    fieldvalue == condition['value']
  when '!='
    fieldvalue != condition['value']
  when 'LIKE'
    fieldvalue.tos.match?(liketoregex(condition['value']))
  end
end

```

```

    else
      raise "Unknown condition operator: {condition['operator']}"
    end
  end
end

def getfieldvalue(field)
  case field
  when 'mergemethod'
    @project.mergemethod.tos
  when 'projectname'
    @project.name
  when 'complianceframework'
    frameworks = @project.compliancemanagementframeworks
    frameworks ? frameworks.map(&:name) : []
  else
    raise "Unknown field: {field}"
  end
end

def liketoregex(pattern)
  regexpattern = Regexp.escape(pattern).gsub('%', '.')
  Regexp.new("^#{regexpattern}$", Regexp::IGNORECASE)
end

end
end
end
end

```

We can then use the above evaluator on the stored JSON expression:

```

ruby
simplequery = {
  "operator" => "=",
  "field" => "mergemethod",
  "value" => "merge"
}

complexquery = {
  "operator" => "AND",
  "conditions" => [
    {
      "field" => "mergemethod",
      "operator" => "=",
      "value" => "merge"
    },
    {
      "operator" => "OR",
      "conditions" => [
        {
          "field" => "projectname",

```

```

      "operator" => "LIKE",
      "value" => "%-team"
    },
    {
      "field" => "complianceframework",
      "operator" => "!=",
      "value" => "SOC2"
    }
  ]
}
]
}

```

```

evaluator = ComplianceManagement::ComplianceRequirement::QueryEvaluator.new(complexquery,
project)
result = evaluator.evaluate Returns true or false

```

```

evaluator = ComplianceManagement::ComplianceRequirement::QueryEvaluator.new(simplequery,
project)
result = evaluator.evaluate Returns true or false

```

We also considered using GitLab Query Language (GLQL), however, since it is not yet mature enough and for the sake of feature velocity we thought of moving forward with the above approach and unify with GLQL later.

External requirements

We would store the external HTTP/HTTPS URLs for the user's external services in the `compliancerequirements` table with 'external' as the `requirementtype`.

We would POST the latest project settings to these external services and expect a boolean status as the response.

We could also create a POST API that can be used to update the status of an external requirement, this would be a similar to setting the status of external status checks.

Workflow

```

mermaid
classDiagram
class namespaces {
  id: bigint
  name: text
  path: text
  ...(more columns)
}
class projects {

```

```

    id: bigint,
    name: text
    path: text
    description: text
    ...(more columns)
}
class compliancemanagementframeworks {
    id: bigint,
    name: text,
    description: text,
    ...(more columns)
}

```

```

class compliancerequirements {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    namespaceid: bigint
    frameworkid: bigint
    name: text
    description: text
    requirementtype: smallint
    externalurl: text
    expression: jsonb
}

```

```

class projectcomplianceconfigurationstatus {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    projectid: bigint
    namespaceid: bigint
    compliancerequirementid: bigint
    status: smallint
}

```

```

class complianceframeworksecuritypolicies {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    frameworkid: bigint
    policyconfigurationid: bigint
    policyindex: smallint
    projectid: bigint
    namespaceid: bigint
}

```

compliancemanagementframeworks --> compliancerequirements : hasmany
 compliancemanagementframeworks <--> projects : manytomany
 compliancerequirements <--> securitypolicyrequirements : hasandbelongstomany

```
projects <-- namespaces : hasmany
namespaces --> compliancemanagementframeworks : hasmany
projects --> projectcomplianceconfigurationstatus : hasmany
compliancerequirements --> projectcomplianceconfigurationstatus : hasone
```

We would drop the existing table `compliancechecks` and update the existing table `compliancerequirements` with the above schema. The `requirementtype` column can either have 'internal' or 'external' as the valid values.

The columns `expression` and `externalurl` are mutually exclusive and depending upon the `requirementtype` one of these columns would be non null. In an unforeseen circumstance where more than one of these columns are filled, we would only honour the column associated with the given requirement type. For example: for a row, if `externalurl` and `expression` both have values but the value of `requirementtype` column is 'internal', we would ignore the value stored in the `externalurl` column and will consider this row as an internal requirement.

We would create a new table `projectcomplianceconfigurationstatus` to store the results of the requirements. Unlike the current implementation we would only store results for the projects that have compliance requirements configured.

We would trigger and re-evaluate the results of all the requirements for a project if there is an update:

1. To a project setting
1. To a group setting that is cascaded to the project
1. To a project's association like compliance management frameworks, labels, topics, etc.
1. To the compliance requirement associated to that project

This means that we need a relationship between `compliancerequirements` and `projectcomplianceconfigurationstatus` database tables.

Whenever there is a change in the status of a requirement we would create an audit event. This will help in capturing the historical changes of that requirement for the given project and would also help in identifying the user and the actions that led to the change in the status of the requirement.

We would query the rows in the `projectcomplianceconfigurationstatus` table for the given namespace and display the results on the compliance dashboard.

Constraints

1. We should limit the maximum number of compliance frameworks a project can have. In the beginning we can keep this limit as 20 and increase later on if needed.
1. We should limit the maximum number of compliance requirements a framework can have. In the beginning we can keep this limit as 50 and increase later on if needed.
1. We should limit the maximum number of fields an expression can have. In the beginning we can keep this limit as 5 and increase later on if needed.
1. We should create an allowlist of project settings and associations that could be used for creating expressions.
1. Without the above limitations it would be very easy for users to abuse our system leading to query timeouts and poor user experience.

Decision

We decided to store the internal compliance requirement expressions as text columns and remove the `compliancechecks` table and only use `compliancerequirements` instead. This would help in reducing redundancy and we could easily display the rows present in `projectcomplianceconfigurationstatus` table on the adherence dashboard.

SECTION: engineering/architecture/design-documents/compliance-frameworks/decisions/004_time_based_triggers.md

Context

As outlined previously, we decided to enqueue Sidekiq workers for processing compliance with framework requirements and their checks. In the previous ADR it was decided to trigger a compliance evaluation when toggling relevant project or group settings, however there are several disadvantages to using such an approach.

Challenge 1: Requirements that do not map directly to settings

Certain requirements do not map 1-1 with project or group settings, for example ensuring all vulnerabilities have been triaged within a 30-day SLO. To perform such a check we need to evaluate a time-based sliding window, which requires enqueueing the compliance evaluation on recurring basis.

Challenge 2: Identifying settings to tie to callbacks

By relying on individual settings we can either (1) evaluate compliance with any settings modification or (2) maintain a

mapping of compliance-relevant settings.

Tying evaluation to any settings modification leads to unnecessary evaluations. Maintaining a setting mapping adds a maintenance burden.

Challenge 3: Application load spikes

By relying on project setting modifications we can experience variable load patterns on our infrastructure, for example updating a group setting with 100,000 projects would lead to 100,000 enqueued evaluations. By relying instead on a time-based syncing mechanism, we can eliminate the burstiness by having more predictable traffic patterns and controlling the concurrency of the evaluation workers.

Decision

Instead of using Sidekiq workers that rely solely on setting modification callbacks, we decided to perform recurring project compliance evaluations via a scheduled cron worker. An initial execution will be performed when a compliance framework is first applied to a project, and then via cron for subsequent evaluations.

The frequency of the cron evaluation worker will initially be set at 12hour intervals (i.e. a given project is evaluated twice a day against each framework to which it's associated).

SECTION: engineering/architecture/design-documents/compliance-frameworks/decisions/005_violations_engine.md

Context

While the adherence report states the current state of a project against the configured framework and requirements/controls, it does not show a history of the state. Most importantly it does not show when the project might not have been adhering and when there was a violation of the requirement/control.

Approach

The violation engine is similar to the configuration checks engine, whenever an audit event is created the system will check that the event does not violate any configured compliance controls.

For example when a Merge Request is merged, we would create an audit event mergerequestmerged, we would then run the the violations engine and check for violations if any. For example: It will check if there is a

control defined for the project
which states all Merge Requests requiring 2 approvers, if the Merge Request has less than 2 then a violation is created from the event.

All GitLab defined controls will have an audit event type configured as its trigger point. We will update the audit event type yml file to include a new parameter that will indicate which control it is associated.

One audit event may have multiple controls associated with it, such as when an MR is merged.

Here is an example audit event type yml file with the new parameter

```
yml
---
name: mergerequestmerged
description: A merge request is merged
introducedbyissue: https://gitlab.com/gitlab-org/gitlab/-/issues/442279
introducedbymr: https://gitlab.com/gitlab-org/gitlab/-/mergerequests/164846
featurecategory: compliancemanagement
milestone: '17.5'
savedtodatabase: true
streamed: true
scope: [Project]
compliance requirement controls: [minimumapprovalsrequired2,
mergerequestpreventcommittersapproval, mergerequestpreventauthorapproval]
```

There are also several controls like vulnerabilityesslodaysoverthreshold, reviewandarchivestalerepos, etc. that cannot be directly linked to an audit event. For these controls, we would rely on FrameworkEvaluationSchedulerWorker. When the worker evaluates these controls and if it finds the status of these controls as failing, we would then create an audit event for the failed control which would then eventually create a violation record.

As per the design violations have a few similarities with work items, like being able to add comments, states like "detected", "resolved". We considered adding violations as a new work item type, however, upon discussing with the 'Plan:Product Planning' group it was decided not to create a new work item type and rather create a separate database table for violations because of the following reasons:

1. Since violations would be generated automatically by our system, this could potentially create a high volume of records and storing them all as work items which is currently a single issues table doesn't seem feasible.
2. The issues table, which stores all work items, has grown to over 300 GB and faces the performance

limitations typical of large database tables.

Due to its size, adding new indexes or columns is prohibited. All feature development must work within these constraints.

3. Violations only had a few similarities with work items and didn't require all the features like assigning users, adding milestones, labels, etc.

We would also have a data retention policy and not store the compliance violations indefinitely. The time for which we would store a violation is not yet finalised but we won't be storing the violations indefinitely.

Design Details

mermaid

classDiagram

class namespaces {

id: bigint

name: text

path: text

...(more columns)

}

class projects {

id: bigint,

name: text

path: text

description: text

...(more columns)

}

class compliancemanagementframeworks {

id: bigint,

name: text,

description: text,

...(more columns)

}

class compliancerequirements {

id: bigint

createdat: timestamp

updatedat: timestamp

namespaceid: bigint

frameworkid: bigint

name: text

description: text

}

class compliancerequirementscontrols {

id: bigint

createdat: timestamp

updatedat: timestamp

namespaceid: bigint

requirementid: bigint

```
name: text
controltype: smallint
externalurl: text
expression: text
encryptedsecrettoken: bytea
encryptedsecrettokeniv: bytea
}
```

```
class projectrequirementcompliancestatus {
  id: bigint
  createdat: timestamp
  updatedat: timestamp
  projectid: bigint
  namespaceid: bigint
  compliancerequirementid: bigint
  status: smallint
}
```

```
class projectcomplianceviolations {
  id: bigint
  createdat: timestamp
  updatedat: timestamp
  projectid: bigint
  namespaceid: bigint
  compliancerequirementscontrolsid: bigint
  auditeventid: bigint
  status: smallint
}
```

```
class auditevents {
  id: bigint
  authorid: bigint
  entityid: bigint
  entitytype: string,
  details: text,
  authorname: text,
  entitypath: text,
  targetdetails: text,
  targettype: text,
  targetid: bigint
  ...(more columns)
}
```

```
class notes {
  id: bigint
  note:
  notabletype:
  authorid: bigint
  projectid: bigint
  ...(more columns)
```

```
}
```

```
class issues {  
  id: bigint  
  title: text  
  projectid: bigint  
  ...(more columns)  
}
```

```
compliancemanagementframeworks --> compliancerequirements : hasmany  
compliancemanagementframeworks <--> projects : manytomany  
compliancerequirements --> compliancerequirementscontrols : hasmany  
projects <-- namespaces : hasmany  
namespaces --> compliancemanagementframeworks : hasmany  
projects --> projectrequirementcompliancestatus : hasmany  
projects --> projectcomplianceviolations : hasmany  
compliancerequirements --> projectrequirementcompliancestatus : hasone  
compliancerequirementscontrols --> projectcomplianceviolations : hasmany  
projectcomplianceviolations --> auditevents : hasone  
projectcomplianceviolations <-- auditevents : hasmany  
projectcomplianceviolations --> notes : hasmany  
projectcomplianceviolations --> issues : hasmany
```

SECTION: engineering/architecture/design-documents/compliance-frameworks/decisions/006_storing_controls_in_a_separate_table.md

Context

As outlined previously, we decided to store the control expression as a json within the compliancerequirements table. In the previous ADR it was decided to allow logical operators such as AND, OR, etc. and create complex nested expressions, however after our current customer and competitor research it was found that we do not need to allow nested expressions and each control within a requirement should be ANDed. This means that a compliance requirement fails if a control fails.

With the previous approach there was also a challenge to evaluate and store the status of individual controls separately, however, on the dashboard the plan is to display the status of each control separately.

Decision

Instead of storing the expression as json within the compliancerequirements table it was decided to create a separate database table named compliancerequirementscontrols. With the introduction of this new database table

we also had to update the existing compliancerequirements table a bit.

mermaid
classDiagram

```
class compliancerequirements {  
  id: bigint  
  createdat: timestamp  
  updatedat: timestamp  
  namespaceid: bigint  
  frameworkid: bigint  
  name: text  
  description: text  
}
```

```
class compliancerequirementscontrols {  
  id: bigint  
  createdat: timestamp  
  updatedat: timestamp  
  namespaceid: bigint  
  requirementid: bigint  
  name: smallint  
  controltype: smallint  
  externalurl: text  
  expression: text  
}
```

```
class projectcontrolcompliancestatuses {  
  id: bigint  
  createdat: timestamp  
  updatedat: timestamp  
  projectid: bigint  
  namespaceid: bigint  
  compliancerequirementid: bigint  
  compliancerequirementscontrolid: bigint  
  status: smallint  
}
```

```
compliancerequirements --> compliancerequirementscontrols : hasmany  
compliancerequirementscontrols --> projectcontrolcompliancestatuses : hasmany
```

We would create schema validators for validating the expression column of the compliancerequirementscontrols table. This columns would contain simple expressions of the following form:

```
json  
{  
  "operator": "=",  
  "field": "minimumapprovalsrequired",
```

```
"value": "2"
}
```

The name field is an enum and would store the enums corresponding to the controls we support, these controls are defined here.

The controltype column can either have 'internal' or 'external' as the valid values.

We would have a unique constraint for compliancerequirementscontrols table with requirementid and name.

This will ensure that a requirement cannot have more than one control of the same name.

Example: A requirement cannot have two rows for 'minimumapprovalsrequired' control.

projectcontrolcompliancestatuses table would also have a unique constraint on compliancerequirementscontrolid and projectid. This will ensure that each row in this table will tell the compliance status of a specific control for a project.

We would limit the maximum number of compliance controls a compliance requirement can have to 5, and increase later on if needed.

The results would be stored in the projectcontrolcompliancestatuses table, however, the status would be at a control level per project instead of a requirement level per project.

The plan for external requirements remains unchanged with the above changes to internal requirements.

1. We should limit the maximum number of fields an expression can have. In the beginning we can keep this limit as 5

and increase later on if needed.

1. We should create an allowlist of project settings and associations that could be used for creating expressions.

1. Without the above limitations it would be very easy for users to abuse our system leading to query timeouts and poor user experience.

SECTION: engineering/architecture/design-documents/compliance-frameworks/decisions/007_external_controls.md

Context

Allow users to create controls to expand on what GitLab offers by default.

External Requirements

- Since we can't reliably predict what kinds of data will be useful for external services, we will initially start out by pinging external services with just project settings attached.
- To avoid complicating role design and needing to control privileges specifically for external controls, we will use the shared HMAC secret for authentication.

Constraints

1. We aim to use the built-in delayed worker execution in Sidekiq to handle defaulting ending status checks to fail after the timeout is hit. In case of an incident where the worker queue list would have to be cleared, we accept that there would be controls in pending state (for longer than the regular timeout) until the next regular schedule triggers a new round of evaluation (currently set at every 12 hours). Worst case should be that status is stale for 11h30m (schedule - regular timeout).

Decision

We decided to let external services post the status of their controls back to us in an async manner, allowing for more time to let them perform more complex checks. This will be especially helpful if they need to request additional data to evaluate (from GitLab itself or other sources).

SECTION: engineering/architecture/design-documents/compliance-frameworks/decisions/008_policy_relationships.md

Context

We need the ability to determine if a security policy exists that satisfies a compliance control.

Option 1

We discussed using a join table between requirements and policies. We also discussed using a join table between controls and policies. Both of these options would accomplish the goal of mapping policies.

The existing structure of security policies -> security controls is stored in YAML, while the structure of compliance requirements -> compliance controls is stored in separate database tables. The misalignment in

storage patterns serves as a warning sign that any implementation connecting the two data structures won't be as simple as we'd like it to be. For instance, updates to security policies will need to also sometimes need to update the matching compliance records.

In this setup, the recurring compliance checks need to confirm these database records to update the status records, and create violations records when appropriate.

Option 2

The second option we discussed was relying on the evaluation engine to verify if a control has a corroborative policy. This would allow us to avoid the complexity of keeping the security and compliance data in sync. This approach doesn't require us to update database records when policies change.

Decision

We decided to rely on the evaluation engine to determine if a security policy exists that satisfies a compliance control. We will not utilize database records to match compliance controls to security policies.

SECTION: engineering/architecture/design-documents/compliance_policy_instance_level_management/_index.md

{{< engineering/design-document-header >}}

Introduction

Compliance frameworks and security policies are currently managed at the group level, which creates challenges for organizations that need to apply consistent compliance and security requirements across multiple groups. This document outlines a proposed architecture to implement Instance Level Compliance and Policy Management functionality using a designated Compliance and Security Policy (CSP) Group approach, allowing organizations to centrally manage and consistently apply compliance frameworks and security policies across the entire instance.

Refer to Instance Level Compliance and Policy Management epic for product requirements.

Proposal

We propose designating a top-level group as the central authority for compliance frameworks and security policies at the instance level. This CSP Group will contain the master version of frameworks and policies, which will then be mirrored to other groups throughout the instance. Top-level group owners can apply these frameworks to their projects but cannot modify them, ensuring consistent application of compliance and security requirements.

Goals

- Allow the application of compliance frameworks and security policies across multiple top-level groups from a central location.
- Enable compliance and security professionals to enforce common requirements across an organization's projects.
- Improve separation of duties for compliance and security management, such as granting only Admin users the ability to designate a CSP group and the users who may manage those policies.
- Simplify the user experience of centralized compliance and policy management by removing the requirement to manage security policy project (SPP) links across groups to a centralized SPP.
- Reduce the need for complex scripting to maintain consistent compliance frameworks across groups.
- Allow for top-level groups to manage policies within their own group while also enabling enforcement of centralized policies across all top-level groups.
- Create a design that can easily evolve when Organization-level scoping becomes available.

Non-Goals

- Replacing existing group-level compliance framework or security policy functionality.
- Building Organization-level management in this phase (though we aim to design for future compatibility).
- Creating new compliance framework types or security policy types.
- Changing the underlying policy evaluation engine.

Terminology/Glossary

- CSP Group: Compliance and Security Policy Group - a designated top-level group with elevated permissions for centrally managing compliance and security policies.
- Mirrored Framework: A read-only copy of a compliance framework from the CSP Group that appears in other groups.
- Framework-scoped Policy: A security policy that targets specific compliance frameworks.
- Original Framework: The source version of a compliance framework in the CSP Group.
- Instance Level: Functionality that applies across the entire GitLab instance.

Design Overview

Core Approach

1. Instance administrator designates a top-level group as the CSP Group.
2. CSP Group admins create compliance frameworks and security policies.
3. Compliance frameworks are automatically mirrored to all other top-level groups in the instance.
4. Security policies are scoped to compliance frameworks.
5. Projects in any group can use the mirrored frameworks.
6. When a project with a mirrored framework runs a pipeline, it enforces associated policies from the CSP Group.

Entity Relationship Diagram

mermaid

erDiagram

Instance ||--o| namespaces : "designates as CSPGroup"

namespaces ||--o{ projects : "hasmany"

namespaces ||--o{ compliancemanagementframeworks : "hasmany"

namespaces ||--o{ compliancerequirements : "hasmany"

projects ||--o| securityorchestrationpolicyconfigurations : "links with"

namespaces ||--o| securityorchestrationpolicyconfigurations : "links with"

securityorchestrationpolicyconfigurations ||--|| projects : "stores policies in"

securityorchestrationpolicyconfigurations ||--o{ securitypolicies : "contains"

securitypolicies ||--||{ complianceframeworksecuritypolicies : "scoped to"

complianceframeworksecuritypolicies }|--|| compliancemanagementframeworks : "scopes"

compliancemanagementframeworks ||--o{ compliancerequirements : "hasmany"

compliancerequirements ||--o{ compliancerequirementscontrols : "hasmany"

compliancemanagementframeworks ||--o{ compliancemanagementframeworks : "is original for"

compliancemanagementframeworks ||--o{ projectcomplianceframeworksettings : "hasmany"

projectcomplianceframeworksettings ||--o{ projects : "manyto many"

projects ||--o{ projectcontrolcompliancestatuses : "hasmany"

compliancerequirementscontrols ||--o{ projectcontrolcompliancestatuses : "hasmany"

projects ||--o{ projectcomplianceviolations : "hasmany"

compliancerequirementscontrols ||--o{ projectcomplianceviolations : "has and belong to many"

projectcontrolcompliancestatuses ||--o{ auditevents : "generates"

compliancerequirements ||--o{ securitypolicyrequirements : "associates with"

securitypolicyrequirements |o--o| securitypolicies : "associates with"

Data Model

Tables

applicationsettings

sql

ALTER TABLE applicationsettings

ADD COLUMN cspnamespaceid BIGINT REFERENCES namespaces(id);

Modified Tables

compliancemanagementframeworks

sql

```
ALTER TABLE compliancemanagementframeworks
ADD COLUMN iscspframework BOOLEAN NOT NULL DEFAULT FALSE,
ADD COLUMN ismirror BOOLEAN NOT NULL DEFAULT FALSE,
ADD COLUMN originalframeworkid BIGINT REFERENCES compliancemanagementframeworks(id),
ADD INDEX(originalframeworkid);
```

complianceframeworksecuritypolicies

sql

```
ALTER TABLE complianceframeworksecuritypolicies
ADD COLUMN isfromcspgroup BOOLEAN NOT NULL DEFAULT FALSE;
```

Core Workflows

CSP Group Designation

1. Instance administrator navigates to Admin Area > Settings > Security and compliance.
2. Administrator selects a top-level group to designate as the CSP Group.
3. System creates an entry in the applicationsettings table.
4. UI updates to show special indicators for the CSP Group.
5. Generate an instance audit event tracking creation, modification, and deletion of CSP Group Designations.

Compliance Framework Creation and Mirroring

1. CSP Group admin creates a compliance framework in the CSP Group.
2. System marks the framework as iscspframework = true.
3. System automatically creates mirrored copies in all other top-level groups.
4. When new top-level groups are created, system creates mirrors for all CSP frameworks.

Security Policy Management

1. CSP Group admin creates security policies in the policy project (.gitlab/security-policies/policy.yml)
2. Admin scopes policies to specific compliance frameworks
3. System creates entries in complianceframeworksecuritypolicies with isfromcspgroup = true

The CSP Group approach supports all security policy types, including scan execution policies, merge request approval policies, and vulnerability management policies. Each policy type follows the same scoping mechanism but has different configuration parameters and enforcement behaviors. Below are examples of different policy types:

Scan execution policy example

yaml

```
scanexecutionpolicy:
  name: Block vulnerable dependencies
```

description: Vulnerability scanning for dependencies
enabled: true
rules:
- type: pipeline
 branches:
 - release/
actions:
- scan: sast
policyscope:
 cspcomplianceframeworks: CSP compliance framework
 - id: 1

Merge request approval policy example

yaml
mergerequestapprovalpolicy:
 name: Security approval for production code changes
 description: Require security team approval for changes to production code
 enabled: true
 rules:
 - type: mergerequest
 branches:
 - main
 - production/
 approvalsrequired: 1
 userapprovers:
 - security-team
 policyscope:
 cspcomplianceframeworks: CSP compliance framework
 - id: 1
 - id: 3

Vulnerability management policy example

yaml
vulnerabilitymanagementpolicy:
 name: Critical vulnerability remediation policy
 description: Sets timelines for addressing critical vulnerabilities
 enabled: true
 rules:
 - type: vulnerability
 severity: critical
 actions:
 - issuecreation:
 duedate: 7 days
 assignee: vulnerabilityauthor
 policyscope:
 cspcomplianceframeworks: CSP compliance framework

- id: 2

Each policy type follows its specific workflow for enforcement:

- Scan execution policies are enforced during CI/CD pipeline execution
- Merge request approval policies are enforced at merge request creation and update
- Vulnerability management policies are triggered when vulnerabilities are detected

Project Compliance Framework Assignment

1. Group owner can assign mirrored frameworks to projects.
2. Group owner can set mirrored frameworks as default for new projects.
3. System creates entries in projectcomplianceframeworksettings referencing the mirrored framework.

Policy Enforcement in CI/CD Pipeline

1. When a project pipeline runs, system checks for assigned compliance frameworks.
 1. If framework is mirrored, system retrieves the original framework from CSP Group.
 1. System retrieves all security policies scoped to the original framework.
 1. Policies are enforced against the project's pipeline.

CSP Group Change Management

Changing CSP Group Designation

1. Instance administrator navigates to Admin Area > Settings > Security and compliance.
 1. Administrator selects a different top-level group to designate as the new CSP Group.
 1. System displays a confirmation dialog with impact information.
 1. Upon confirmation:
 1. System updates the entry in applicationsettings table.
 1. System marks all frameworks in the old CSP Group as iscspframework = false.
 1. System marks all frameworks in the new CSP Group as iscspframework = true.
 1. All mirrored frameworks linked to the old CSP Group are deleted across the instance.
 1. All mirrored frameworks linked to the old CSP Group are removed from projects.
 1. All security policies linked to the old CSP Group are unlinked.
 1. New frameworks from the new CSP Group are mirrored to all top-level groups.
 1. System generates an audit event tracking the CSP Group change.
 1. UI indicators update to reflect the new CSP Group.

Removing CSP Group Designation

1. Instance administrator navigates to Admin Area > Settings > Security and compliance.
 1. Administrator selects "Remove CSP Group designation".
 1. System displays a confirmation dialog with impact information.
 1. Upon confirmation:
 1. System removes the entry from applicationsettings table.
 1. System marks all frameworks in the CSP Group as iscspframework = false.
 1. All mirrored frameworks linked to the CSP Group are deleted across the instance.

1. All mirrored frameworks linked to the CSP Group are removed from projects.
1. System generates an audit event tracking the CSP Group removal.
1. UI indicators are removed to reflect the absence of a CSP Group.

Compliance Framework Change Management

Modifying a Compliance Framework in CSP Group

1. CSP Group admin edits a compliance framework in the CSP Group.
1. System triggers CSP::FrameworkUpdatePropagationService.
1. Service finds all mirrored frameworks associated with the original.
1. Service updates all mirrored frameworks with the new information.
1. System generates audit events for each update.
1. UI updates to show the updated framework information across all groups.

Deleting a Compliance Framework in CSP Group

1. CSP Group admin deletes a compliance framework in the CSP Group.
1. System displays a confirmation dialog with impact information.
1. Upon confirmation:
 1. System marks the framework as deleted in the database.
 1. System triggers CSP::SyncDeletedFrameworksJob.
 1. Any projects that had the deleted framework assigned have that framework unassigned.
 1. Job deletes all mirrored frameworks associated with the original.
 1. System generates audit events for each deletion.

Tracking progress of propagation

When a compliance framework in the CSP Group is modified, it may take some time for the changes to fully propagate to all mirrored frameworks, especially in instances with many top-level groups. To enhance the user experience, we can track propagation progress by calculating the total number of frameworks to be mirrored in advance and updating the status as each one is completed.

Security Policy Change Management

Modifying a Security Policy in CSP Group

1. CSP Group admin edits a security policy in the policy project.
1. System updates the policy yaml.
1. Changes take effect immediately for all projects using frameworks associated with that policy.
1. No action is needed on mirrored frameworks as policy enforcement looks up the original framework.
1. System generates an audit event tracking the policy change.

Removing a Security Policy in CSP Group

1. CSP Group admin removes a security policy from the policy project.
1. System updates the policy configuration.

1. Policy is immediately removed from enforcement across all projects.
1. System generates an audit event tracking the policy removal.

Top-Level Group Management

Adding a New Top-Level Group

1. User creates a new top-level group.
1. System triggers CSP::MirrorComplianceFrameworksJob for the new group.
1. Job creates mirrors of all CSP Group frameworks in the new group.
1. Group owners can immediately assign these frameworks to projects.

Deleting a Top-Level Group

1. User deletes a top-level group.
1. System performs standard group deletion operations.
1. All mirrored frameworks in that group are deleted as part of group deletion.
1. No special handling is needed beyond standard group deletion.

API Design

GraphQL Extensions

```
graphql
type Query {
  instanceCspGroup: Namespace
  complianceManagementFrameworks(
    namespaceId: ID!,
    isMirror: Boolean,
    isCspFramework: Boolean
  ): [ComplianceManagementFramework!]!
}

type Mutation {
  designateInstanceCspGroup(namespaceId: ID!): Namespace
  removeCspGroupDesignation: Boolean
  createCspComplianceFramework(input: FrameworkInput!): ComplianceManagementFramework
  updateCspComplianceFramework(id: ID!, input: FrameworkInput!): ComplianceManagementFramework
  deleteCspComplianceFramework(id: ID!): Boolean
  scopePolicyToFramework(policyId: ID!, frameworkId: ID!): ComplianceFrameworkSecurityPolicy
  assignFrameworkToProject(projectId: ID!, frameworkId: ID!): ProjectComplianceFrameworkSetting
  setDefaultGroupFramework(namespaceId: ID!, frameworkId: ID!): Namespace
}

type ComplianceManagementFramework {
  isCspFramework: Boolean!
  isMirror: Boolean!
  originalFramework: ComplianceManagementFramework
  mirroredFrameworks: [ComplianceManagementFramework!]!
  scopedPolicies: [SecurityPolicy!]!
```

}

```
extend type Namespace {  
  cspFrameworks: [ComplianceManagementFramework!]!  
  defaultFramework: ComplianceManagementFramework  
  isCspGroup: Boolean!  
}
```

```
extend type Project {  
  assignedFrameworks: [ComplianceManagementFramework!]!  
}
```

Authorization Model

Permission Structure

CSP Group-Related Permissions

- manageinstancecspgroup: Designate/undesignate CSP Group (instance admins only).
- managecspcomplianceframeworks: Create/edit frameworks in CSP Group (CSP Group admins).
- managecspsecuritypolicies: Create/edit policies in CSP Group (CSP Group admins).
- scopecsp policiestoframeworks: Associate policies with frameworks (CSP Group admins).

Group-Related Permissions

- assignmirroredframeworks: Assign mirrored frameworks to projects (group owners/maintainers).
- setdefaultmirroredframeworks: Set default frameworks for group (group owners).

Permission Matrix

| Role | Designate CSP | Manage CSP Frameworks | Manage CSP Policies | Assign Mirrored Frameworks | Set Default Frameworks |

|-----|-----|-----|-----|-----|

--|-----|

| Instance Admin | . | . | . | . | . |

| CSP Group Admin | . | . | . | . | . |

| CSP Group Maintainer | . | . | . | . | . |

| Group Owner | . | . | . | . | . |

| Group Maintainer | . | . | . | . | . |

| Developer | . | . | . | . | . |

User Interfaces

CSP Group Management UI

- Instance Settings > Compliance page with CSP Group designation.
- Visual indicator for CSP Group in groups list.
- Special header in CSP Group pages indicating its role.

CSP Framework Management UI

- Enhanced compliance framework UI in CSP Group.
- Framework creation/editing forms.

CSP Policy Management UI

- Policy editor in CSP
- Security Policy Project in CSP

Group Policies UI (Secure > Policies)

- Group users may view policies applied to their group, including those enforced via CSP

Project Policies UI (Secure > Policies)

- Project users may view policies applied to their project, including those enforced via CSP

Group Framework UI

- List of available mirrored frameworks.
- Framework assignment UI for projects.
- Default framework settings for group.
- Visual indicators showing frameworks are mirrored (read-only).

Project Compliance UI

- Assigned frameworks display.
- Framework assignment selector.
- Pipeline integration showing applied policies.

Technical Implementation

Models and Associations

These models define the structure of the compliance framework mirroring system.

ComplianceManagementFramework

- This model represents a compliance framework, which consists of a set of security policies, requirements, and rules that organizations must follow.
- Defines compliance frameworks that belong to a namespace (group).
- Includes mirrored framework functionality with `ismirror` and `originalframeworkid` fields.
- Establishes relationships with:
 - Mirrored frameworks (so that it knows which groups have copies)
 - Compliance framework security policies (for applying security policies)
 - Security policies (via `complianceframeworksecuritypolicies`).
- Defines a scopes:
 - `cspframeworks`: filters frameworks that are designated as CSP frameworks.
 - `mirrored`: filters frameworks that are mirrors of CSP frameworks.

- originals: filters frameworks that are not mirrors.

Services

CSP::FrameworkMirroringService

- Ensures that compliance frameworks created in the CSP group are propagated to other groups automatically.
- Initializes with a CSP group ID.
- Returns an error if no CSP group is set (to prevent execution without context).
- Retrieves all compliance frameworks in the CSP group.
- Identifies all top-level groups (excluding the CSP group).
- Creates a mirrored compliance framework in each top-level group for each CSP framework by:
 - Setting ismirror = true
 - Setting originalframeworkid to the ID of the source framework
 - Copying relevant attributes from the original

CSP::FrameworkUpdatePropagationService

- New service that propagates updates from original frameworks to their mirrors.
- When an original framework is updated, finds all mirrors and updates their attributes.
- Maintains consistency between originals and mirrors.

CSP::PolicyResolverService

- Provides a way to resolve and return policies that apply to a specific project.
- Retrieves policies for a given project and returns them as a structured response.

Background Jobs

These are asynchronous workers that run in the background to handle framework mirroring and cleanup.

CSP::MirrorComplianceFrameworksJob

- Automates the initial framework mirroring process without requiring manual execution.
- Retrieves the CSP group ID from InstanceSettings.
- Runs CSP::FrameworkMirroringService to mirror compliance frameworks across top-level groups.

CSP::SyncDeletedFrameworksJob

- Prevents stale mirrored frameworks from existing when the original compliance framework is deleted.
- Finds frameworks where ismirror = true and originalframeworkid points to a non-existent framework.
- Deletes these orphaned mirrored frameworks.

Controllers

Controllers expose API endpoints for managing the CSP group configuration.

API::V4::Instance::CspGroupController

- Provides an API to retrieve and update the CSP group that holds the source compliance frameworks.
- Authentication & Authorization
 - Requires user authentication.
 - Ensures only admins can update the CSP group.
- Show Action
 - Retrieves and returns the CSP group namespace ID.
- Update Action
 - Updates the CSP group namespace in InstanceSettings.
 - Triggers CSP::FrameworkMirroringService to immediately mirror compliance frameworks to other groups.
 - Returns a success response.

Future Considerations

Organization Entity Integration

The mirroring approach provides a clean path to organization-level implementation:

- Replace instance-level designation with organization entity.
- Convert mirroring system to work across organizations.
- Maintain the same permission model but at organization level.

Extended Capabilities

- Exclusion requests for specific projects.
- Detailed compliance reporting.
- Framework version control and history.
- Enhanced policy templating.
- Automated compliance violation remediation.
- Compliance dashboard for instance-wide visibility.

SECTION: engineering/architecture/design-
documents/composable_codebase_using_rails_engines/_index.md

{{< engineering/design-document-header >}}

NOTE:

This architecture design document has been superseded by GitLab Modular Monolith.

<!-- TODO: change to new design doc URL -->

The one of the major risks of a single codebase is an infinite growth of the whole application. The more code being added results in not only ever increasing resource requirements

for running the application, but increased application coupling and explosion of the complexity.

Executive summary

This blueprint discusses an impact of introducing Application Layers as a way to reduce and improve the application codebase. This discusses the positive and negative outcomes of the proposed solution and tries to estimate the impact on GitLab.com and smaller installations.

Application Layers tries to split GitLab Rails codebase horizontally following the pattern of how we actually run GitLab instead of vertical split. This follows the idea that a single feature needs to run in many different ways (CI for example has Web interface, uses API, and performs background processing), and we are not able to easily run only a given feature separate to the rest application (like CI) due to coupling.

The proposal itself does allow us to disconnect some aspects of the features. These aspects could be treated as components that are run separately from the rest of the stack, but still sharing a large portion of core. This model could be implemented to provide an API interface for external tooling (Runners API, Packages API, Feature Flags Unleash API) and allow us to have much better resiliency and much easier way to scale application in the future.

The actual split was tested with the usage of Rails Engines implementing separate gems in a single repository. The Rails Engines allowed us to well describe the individual components with its dependencies and run an application consisting of many Rails Engines.

The blueprint aims to retain all key aspects of GitLab success: single and monolithic codebase (with a single data-store), but allows us to better model application and make our codebase more composable.

Challenges of the Monolith (a current state)

Today the usage of monolith proves to be challenging in many cases. A single big monolith codebase without clear boundaries results in a number of problems and inefficiencies, some of them being:

- Deep coupling makes application harder to develop in longer term, as it leads to a spaghetti implementation
 - instead of considering building more interface-based architecture
- Deep coupling between parts of the codebase making it harder to test. To test only a small portion of application
 - we usually need to run a whole test suite to confidently know which parts are affected. This to some extent can be improved by building a heuristic to aid this process, but it is prone to errors and hard

to keep accurate at all times

- All components need to be loaded at all times to run only parts of the application
- Increased resource usage, as we load parts of the application that are rarely used in a given context
- The high memory usage results in slowing the whole application as it increases GC cycles duration
 - creating significantly longer latency for processing requests or worse cache usage of CPUs
- Increased application boot-up times as we need to load and parse significantly more files
- Longer boot-up times slows down the development, as running application or tests takes significantly longer
 - reducing velocity and amount of iterations

Composable codebase dimensions

In general, we can think about two ways how codebase can be modeled:

- vertically in Bounded Contexts, each representing a domain of the application, ex.: All features related to CI are in a given context
- horizontally in Application Layers: Sidekiq, GraphQL, REST API, Web Controllers, all Domain Models and Services that interface with DB directly

This blueprint explicitly talks about horizontal split and Application Layers.

Current state of Bounded Contexts (vertical split)

The Bounded Contexts is a topic that was discussed extensively number of times for a couple of years.

Reflected in number of issues:

- Create new models / classes in a module / namespace
- Make teams to be maintainers of their code
- Use nested structure to organize CI classes
- WIP: Make it simple to build and use "Decoupled Services"

We are partially executing a Bounded Contexts idea:

- Make each team to own their own namespace, namespace which is defined as a module in a codebase
- Make each team to own their own tests, as namespaces would define a clear boundaries
- Since we use namespaces, individual contributor or reviewer can know who to reach from domain experts about help with the given context

The module namespaces are actively being used today to model codebase around team boundaries. The most prominent namespaces being used today are `Ci::` and `Packages::`. They provide a good way to contain the code owned by a group in a well-defined structure.

However, the Bounded Contexts while it helps development, it does not help with the above

stated goals. This is purely a logical split of the code. This does not prevent deep-coupling. It is still possible to create a circular dependency (and it often happens) between a background processing of CI pipeline and Runner API interface. API can call Sidekiq Worker, Sidekiq can use API to create an endpoint path.

The Bounded Contexts do not make our codebase smarter to know what depends on what, as the whole codebase is treated as single package that needs to be loaded and executed.

Possible additional considerations to the disadvantages of Bounded Context:

- It can lead to tribal knowledge and duplicate code
- The deep coupling can make it difficult to iterate and make minimal changes
- Changes may have cascading effects that are difficult to isolate due to the vertical split

The Application Layers (horizontal split)

While we continue leveraging Bounded Contexts in form of namespace separation that aids development and review process the Application Layers can provide a way to create a clean separation between different functional parts.

Our main codebase (GitLab Rails after a GitLab running on Ruby on Rails) consists many of implicit Application Layers.

There are no clear boundaries between each layer which results in a deep coupling.

The concept of Application Layers looks at the application from the perspective of how we run the application

instead of perspective of individual features (like CI or Packages). GitLab application today can be decomposed into the following application layers. This list is not exhaustive, but shows a general list of the different parts of a single monolithic codebase:

- Web Controllers: process Web requests coming from users visiting web interface
- Web API: API calls coming from th